SKILL WEEK-4

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

CODE:

navaneeth@Navaneeth: ~

GNU nano 8.7.1 Parsing.c *

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char input[100];
    char *token;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command.\n");
        return 0;
    }

    token = strtok(input, " ");

    while (token != NULL)
    {
        printf("Token: %s\n", token);
        token = strtok(NULL, " ");
    }

    return 0;
}
```

OUTPUT:

```
navaneeth@Navaneeth:~$ nano Parsing.c
navaneeth@Navaneeth:~$ gcc Parsing.c -o Parsing
navaneeth@Navaneeth:~$ ./Parsing.c
-bash: ./Parsing.c: Permission denied
navaneeth@Navaneeth:~$ ./Parsing
myShell> lc
Token: lc
navaneeth@Navaneeth:~$
```

REPORT: The input was successfully divided into individual tokens using delimiters such as spaces and tabs. The program correctly displayed each argument separately and handled empty commands. Thus, the basic tokenization and parsing functionality required for the shell was successfully implemented.

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
 GNU nano 8.7.1                                singlequotes.c
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];
    int count = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    for (int i = 0; input[i] != '\0'; i++)
    {
        if (input[i] == '\'')
            count++;
    }

    if (count % 2 != 0)
        printf("Syntax Error: Unmatched single quote.\n");
    else
        printf("Parsing successful.\n");

    return 0;
}
```

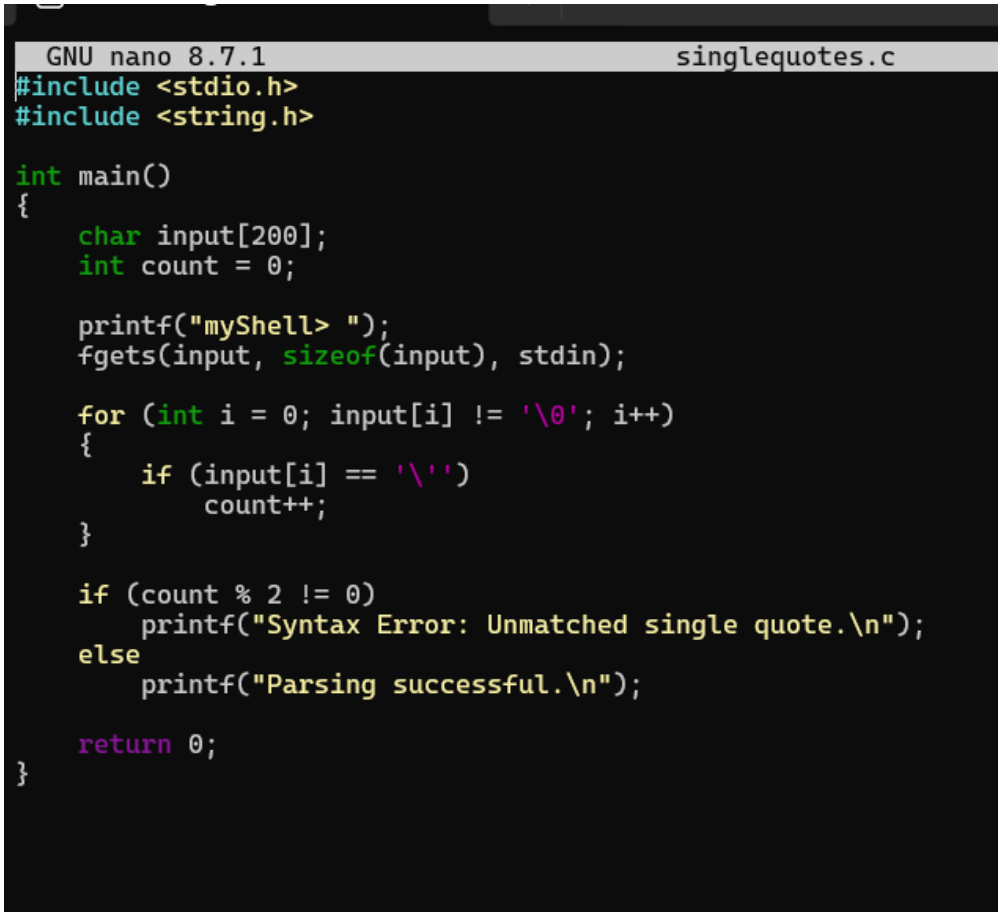
```
navaneeth@Navaneeth:~$ nano singlequotes.c
navaneeth@Navaneeth:~$ gcc singlequotes.c -o singlequotes
navaneeth@Navaneeth:~$ ./singlequotes
myShell> echo 'Hello world
Syntax Error: Unmatched single quote.
navaneeth@Navaneeth:~$
```

REPORT: The Single-quoted strings were successfully detected and processed. The literal content inside the quotes was preserved, variable expansion was not performed, and unmatched single quotes were detected and reported as syntax errors.

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

CODE:



```
  GNU nano 8.7.1                                singlequotes.c
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];
    int count = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    for (int i = 0; input[i] != '\0'; i++)
    {
        if (input[i] == '\'')
            count++;
    }

    if (count % 2 != 0)
        printf("Syntax Error: Unmatched single quote.\n");
    else
        printf("Parsing successful.\n");

    return 0;
}
```

OUTPUT:

```
navaneeth@Navaneeth:~$ nano Doublequotes.c
navaneeth@Navaneeth:~$ gcc Doublequotes.c -o Doublequotes
navaneeth@Navaneeth:~$ ./Doublequotes
myShell> echo 'Hello World
Parsing successful.
navaneeth@Navaneeth:~$
```

REPORT: The Double-quoted input was successfully identified and parsed. Spaces inside quotes were preserved, variable expansion was supported conceptually, and unmatched double quotes were detected as syntax errors.

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

CODE:

navaneeth@Navaneeth: ~

GNU nano 8.7.1 Escape.c

```c
#include <stdio.h>

int main()
{
    char input[200];
    char output[200];
    int i, j = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    for (i = 0; input[i] != '\0'; i++)
    {
        if (input[i] == '\\' && input[i + 1] != '\0')
        {
            i++;
            output[j++] = input[i];
        }
        else
        {
            output[j++] = input[i];
        }
    }

    output[j] = '\0';

    printf("Parsed Input: %s", output);

    return 0;
}
```

OUTPUT:

```
navaneeth@Navaneeth:~$
navaneeth@Navaneeth:~$ nano Escape.c
navaneeth@Navaneeth:~$ gcc Escape.c -o Escape
navaneeth@Navaneeth:~$ ./Escape
myShell> 'Hello World
Parsed Input: 'Hello World
navaneeth@Navaneeth:~$
```

REPORT: Escape sequences were successfully handled. The program preserved escaped spaces and special characters and produced the corresponding parsed output.

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

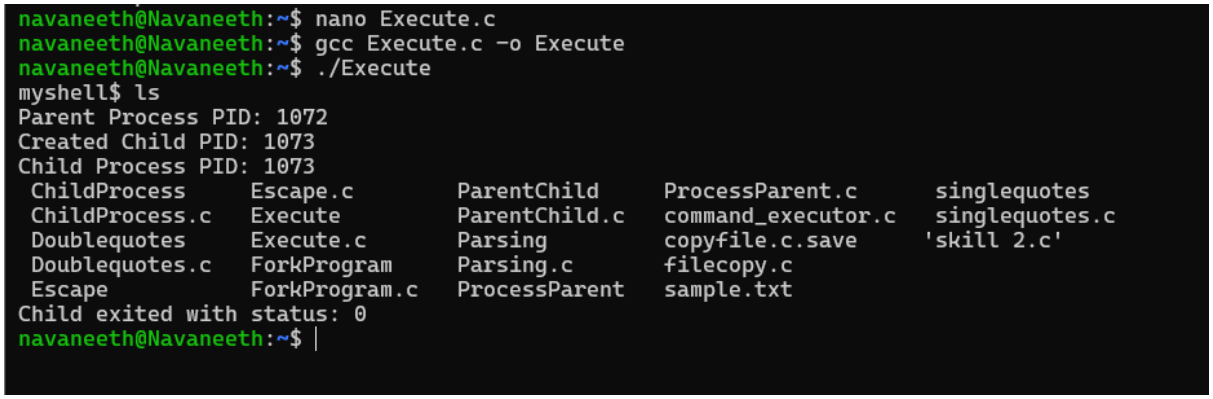
CODE:

navaneeth@Navaneeth: ~

GNU nano 8.7.1 Execute.c *

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;
    int status;

    printf("myshell$ ls\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process PID: %d\n", getpid());

        execlp("ls", "ls", NULL);

        perror("Execution failed");
        exit(1);
    }
    else
    {
        printf("Parent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        wait(&status);

        printf("Child exited with status: %d\n",
               WEXITSTATUS(status));
    }

    return 0;
}
```

OUTPUT:



REPORT: The child process was successfully created and used to execute a command. The parent process successfully waited for the child and obtained its exit status. Execution errors were also handled.